

Supplementary III

(Wagner 128-Sum 64-bit Hash Summation Algorithm)

Based on the standard Wagner generalized birthday algorithm, this project independently develops two C language programs. All implementations generate 128 SHA-1 hash values, whose summation under 64-bit modulo operation equals 0, and an arbitrary custom 64-bit constant respectively.

Multiple repeated experiments are conducted to record the number of program reruns required for each valid group, and the running time of each single execution. Actual running screenshots and typical terminal output logs are also attached.

1 Code File Description

1. **Test0.c:** The 64-bit modulo sum of 128 hash values = 0.
2. **TestH.c:** The 64-bit modulo sum equals any customized 64-bit random value

Unified Algorithm Parameters:

- Initial table number: 128
- Merging layers: 7 layers ($\log_2 128$)
- Matching bits per layer: 8 bit
- Capacity of each table: 256
- Hash function: SHA-1; only the lower 64 bits are used for modulo 2^{64} addition
- Algorithm type: Probabilistic birthday search algorithm

2 Compilation & Runtime Environment

2.1 Dependencies

- GMP Library: Arbitrary-precision integer arithmetic
- OpenSSL Library: SHA-1 hash computation

2.2 Compilation Command

```
gcc -o Test0 Test0.c -lgmp -lcrypto
gcc -o TestH TestH.c -lgmp -lcrypto
```

2.3 Execution Command

```
./Test0
./TestH
```

3 Test Scheme

1. All two programs are tested independently, with 10 valid result groups collected for each.
2. Evaluation Metrics:
 - Number of program reruns required to obtain a valid group;
 - Execution time of each single program run.
3. Time Performance: The execution time of each run is stably maintained within 1s ~4s, with minor fluctuations depending on hardware configuration.
4. Algorithm Characteristics: The Wagner algorithm is probabilistic. It may fail to find a valid solution in a single run, and rerunning the program is a normal phenomenon consistent with theoretical properties.

4 Experiment Results

4.1 Test0.c(Sum=0)

Number of reruns required for 0 valid groups:

1, 13, 2, 6, 3, 18, 7, 5, 17, 3

4.2 TestH.c(Sum=Custom 64-bit Value)

Number of reruns required for 10 valid groups:

5, 3, 2, 1, 18, 5, 3, 11, 2, 18

4.3 Overall Time Performance

All three programs run stably between 1s ~4s per execution. The target value exhibits no obvious influence on runtime efficiency and search success rate.

5 Program Running Screenshot

5.1 Successful Execution Interface

```

✓ Total execution time: 3.465 seconds

🎉 SUCCESS: Found 128 hashes with FULL 64-bit sum = 0!

=== 128 Original Seeds ===
9835205400154 19586184130334 10842673320646 4040509080773 10
300798642960 2158787443173 14723278903515 4809751023592 9622
0805656 9299302906070 15439242286340 2468003446617 295317856
586 2427493398889 4798809779172 6652872119270 3957997469564
12018444511686 8544970021975 13405189952221 17239884182477
16830290944361 6778955534657 1209199474942 11733624945112 48
4276377368 1190150997492 13058514167859 7227486407938 529444
8270098 3424692660373 1814798340490 1560816410784 4101572231
018 2376264653081 17744174723378 15219295693684 188826731385
42119475857 4804014196161 18833297566436 10109720466806 1444
08819 19483278129140 14759741229408 17539342809613 209878451
21677 7578738851822 10528132502053 2451452292342 14649324162

=== Final Verification ===
Full 64-bit sum: 0x0000000000000000
Full sum (hex): 0
spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-

```

```

spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14
=== Wagner 128-Sum (64-bit Target) ===
The target random value is 0x123456789abcdef0
The highest 8-bit of the target value: 0x12
The lowest 56-bit of the target value: 0x3456789abcdef0
K=128, LIST_SIZE=256
✓ Level 0 is finished
✓ Level 1 is finished
✓ Level 2 is finished
✓ Level 3 is finished
✓ Level 4 is finished
✓ Level 5 is finished
✓ Level 6 is finished

The elements of the root list: 8
Searching for the target: 0x123456789abcdef0
The lowest 56-bit 0x3456789abcdef0

The execution time of the algorithm is 1.584 seconds

🎉 SUCCESS: Found 128 hashes with FULL 64-bit sum is any constant!
The total result is: 0x123456789abcdef0
spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14

```

5.2 Typical Terminal Output

128 seeds, the sum of whose hash values is 0:

```
spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14/include$ ./Test0
Original Wagner 128-sum = 0 (64-bit strict)
Parameters: K=128, LIST_SIZE=256, BASE_BITS=8
```

```
=== Searching root list for 64-bit zero ===
```

```
Root list size: 29 elements
```

```
Candidate 0: 0xd600000000000000
```

```
...
```

```
Candidate 27: 0x0000000000000000
```

```
Total execution time: 3.053 seconds
```

```
SUCCESS: Found 128 hashes with FULL 64-bit sum = 0!
```

```
=== 128 Original Seeds ===
```

```
19065576510235 552142260428 12288526990590 13691113730778 21111797121263
4762110351450 9200643031587 4306505181907 8161704252285 7330095482346
18437951572778 5099168712872 18938341273288 5526691193522 2009824633784
18378715343962 1643961544219 11021737574565 1967587674732 12970673044910
10489743235242 3349951475435 10269316515694 14309842515954 5962689336270
7847069096426 11735156976717 14090068357095 15684429717314 8336874317501
15402930987839 17256117818044 3630052858339 12917102198484 9948262638860
19930029978967 4795835459313 11989070689677 20019447239820 9529162150172
12027792380446 5189832420502 11294042910927 13898649511037 15829980841462
20806893231646 2508130251980 6118058942229 17801597982414 10103405532681
7960076542931 19303098433292 739063303473 19731073493630 9821399784043
7376825854126 18153396714458 3704915124705 11991408444944 15506117325137
12026515825576 2321673375679 12088090106114 12293312006195 3183260736597
18235515046675 640574727011 70974767253 14454098087651 13302078427683
15627886578136 5924209928407 18861160108557 17768203938906 19603765629156
15247148449355 15534450409701 14161678289964 9101302190175 13710476820424
10586663270549 18867933220887 4249723280993 19072644851500 16206700311627
9180542011866 1120753792153 20810006432510 6546880222660 9867095783016
13641065863197 761436693495 17048769313759 10631955613825 16768473544069
7406412054534 14396065094703 20479692674901 2493792455309 2170799135408
3935832345691 5316690595872 638585906263 12848213736417 13932803506772
20765710517045 10626612327229 1317349667593 11356288767675 15989707108115
7758428788192 4348645458598 2693966918900 5016403109174 7746556069194
2020391329466 9950896079838 12616682969976 3441559750431 14241237980703
8578563300867 5731893151057 15548194721287 8231578371731 5582263871768
13995668102086 19867632462261 6181043982581
```

```

=== Final Verification ===
Full 64-bit sum: 0x0000000000000000
Full sum (hex): 0

```

128 seeds, the sum of whose hash values is any customized constant.

```

spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14/include$ ./TestH
=== Wagner 128-Sum (64-bit Target) ===
The target random value is 0x123456789abcdef0
The highest 8-bit of the target value: 0x12
The lowest 56-bit of the target value: 0x3456789abcdef0
K=128, LIST_SIZE=256
Level 0 is finished
Level 1 is finished
Level 2 is finished
Level 3 is finished
Level 4 is finished
Level 5 is finished
Level 6 is finished

The elements of the root list: 50
Searching for the target: 0x123456789abcdef0
The lowest 56-bit 0x3456789abcdef0

The execution time of the algorithm is 1.716 seconds

SUCCESS: Found 128 hashes with FULL 64-bit sum = 0!
The total result is: 0x123456789abcdef0

```

If no valid result is found in one execution, the terminal prompts:

```

Candidate 14: 0xae00000000000000
Candidate 15: 0x3b00000000000000
Candidate 16: 0x9f00000000000000
Candidate 17: 0xe100000000000000
Candidate 18: 0xf900000000000000
Candidate 19: 0xa400000000000000

✅ Total execution time: 3.200 seconds

❌ No 64-bit zero found in this run
This is normal (probability ~63% per run), just run again
spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14/include$ ./Test0

```

A new search will start after program restart.

6 Result Analysis

6.1 Algorithm Correctness

All both implemented programs can stably generate 128 random seeds satisfying the constraints. The lower 64-bit sum of the corresponding SHA-1 hash values strictly equals the predefined target, which verifies the correctness of the Wagner algorithm implementation.

6.2 Probability Rationality

The distribution of rerun times is consistent with the theoretical success probability of the Wagner birthday search algorithm, without abnormal deviation.

6.3 Stable Time Performance

Each program run takes 1 to 4 seconds with controllable computational overhead. All both versions achieve consistent performance under different target values.

6.4 Good Generality

TestH.c supports arbitrary 64-bit target configuration and maintains the same success rate and time overhead as the fixed-target versions (sum = 0 / sum = 1). It can be directly applied to various cryptography simulation scenarios.

7 Notes

1. No code modification is required if the search fails; simply rerun the program.
2. A runtime of 1s ~ 4s is normal; slight fluctuations are caused by CPU hardware differences.
3. All programs compile without warnings and run without memory leaks, suitable for cryptography simulation and function invocation.
4. The target summation value can be customized by modifying the TARGET macro in TestH.c.